D-SVM Control Chart

Algorithm Technical Documentation

D-SVM Control Chart for Multivariate Process Monitoring

Language: Python
Algorithm: D-SVM Control Chart
Date: March 2026

1. Overview

This document describes the algorithmic implementation of the D-SVM Control Chart, a multivariate statistical process monitoring method. The algorithm frames process monitoring as a two-class classification problem: a reference set of historical in-control observations is labelled as the negative class, and a moving window of recent test observations is labelled as the positive class. A binary SVM is fitted to this combined labelled dataset at each monitoring step, and the sigmoid-transformed mean decision value of the window observations is used as the monitoring statistic D_SVM.

The control limit is calibrated offline via a double bootstrap procedure applied to a held-out portion of the bootstrapped training data. During online monitoring, the SVM is refit at every step; no adaptive update of the reference set is performed.

Module	Role
Data ingestion and normalisation	Load train/test CSVs; apply joint RobustScaler normalisation
Reference set construction	Select representative training subset via R-column quantile banding; random subsample to S0
Bootstrap sample generation	Draw 1,000 bootstrap rows from the selected S0 to create a held-out calibration pool
Moving window construction	Prepend tail of S0 to the test sequence; define label vectors for SVM training
Bootstrap control limit calibration	Estimate the control limit cl_final by applying the monitoring statistic to bootstrap windows over 100 outer replicates

Module	Role
Online monitoring loop	Iterate over test observations: refit SVM on reference + window, compute D_SVM, compare to control limit
Alarm detection and visualisation	Collect alarm indices; plot D_SVM against the fixed control limit

2. Data Ingestion and Normalisation

2.1 Input Files

The pipeline requires two CSV files: `real example-testdata.csv` and `real example-traindata.csv`. The first column of each file is dropped via `.iloc[0:n, 1:]`. The column `Confirmedcases` is separately extracted from the training file as the one-dimensional array `R`, which is used exclusively as the criterion for reference set selection.

Variable	Source	Shape	Role
`y_test`	`real example-testdata.csv`	(n_test, p)	Observations to be monitored online
`y_train`	`real example-traindata.csv`	(n_train, p)	Historical reference data for selection and SVM fitting
`R`	Column Confirmedcases of traindata	(n_train,)	Univariate selection criterion for IQR banding

2.2 Joint Robust Normalisation

Training and test data are concatenated before scaling so that a single transformation is estimated from the combined dataset:

```
All = np.r_[y_test, y_train]
robust = preprocessing.RobustScaler()
All = robust.fit_transform(All)
y_test  = All[:y_test.shape[0], :]
y_train = All[y_test.shape[0]:, :]
```

3. Reference Set Construction and Bootstrap Sample

3.1 Quantile Banding on R

Observations whose `R` value falls in the band between the 35th and 50th percentiles of the training `R` distribution are selected as candidates for the reference set:

```
q1 = np.quantile(R, 0.35)
```

```
q3 = np.quantile(R, 0.50)
IQR_ind = np.intersect1d(np.where(R >= q1)[0], np.where(R <= q3)[0])
y_IQR = y_train[IQR_ind, :]
```

3.2 Random Subsample to S0

Row indices of `y_IQR` are shuffled uniformly and the first `S0 = 100` indices are used to subset `y_train`:

```
row_rand_array = np.arange(y_IQR.shape[0])
np.random.shuffle(row_rand_array)
extract = row_rand_array[0:S0]
y_train = y_train[extract, :]   # overwritten with the 100 selected rows
y_S0 = y_train[:S0, :]          # reference set: shape (100, p)
```

Note: Unlike the Time-Adaptive SVDD pipeline, this implementation does not apply `abs(np.sort(-...))` to sort the selected indices; the rows are taken in the original shuffled order. `y_train` is overwritten in place and is used exclusively as the reference set from this point onward.

3.3 Bootstrap Sample Generation

A bootstrap sample of 1,000 rows is drawn with replacement from the 100-row reference set and stored as `y_train_boot`. This bootstrap sample is used in the control limit calibration step and is not involved in online monitoring:

```
index = list(range(S0))                                    # [0, 1, ..., 99]
bootstrap_index = np.random.choice(index, 1000, replace=True) # 1000 row indices
y_train_boot = y_train[bootstrap_index, :]                 # shape: (1000, p)
```

The rows of `y_train_boot` beyond the first `S0 = 100` rows are used as the moving calibration pool during bootstrap control limit estimation. The first 100 rows overlap exactly with `y_S0`; the remaining 900 rows form the calibration pool `y_move = y_train_boot[S0:, :]` used in the outer bootstrap loop.

4. Moving Window and SVM Label Vector

4.1 Moving Window Construction

The test sequence `X = y_test` is prepended with the last `Movewindow - 1 = 4` rows of `y_S0` to form the extended sequence `X_w`, ensuring that the first test observation can be evaluated within a full window of length `Movewindow = 5`:

```
Movewindow = 5
X_w = np.r_[y_S0[y_S0.shape[0]-Movewindow+1:, :], X]
# X_w shape: (n_test + Movewindow - 1, p)
```

4.2 SVM Label Vector

The binary SVM requires a label vector for the combined dataset `y_combine = np.r_[y_S0, New_X_w]`. The label vector `y_final` assigns class −1 to the `S0 = 100` reference observations and class +1 to the `Movewindow = 5` window observations:

```
y_n0    = -1 * np.ones(y_S0.shape[0])   # shape (100,): reference class = -1
y_nw    =  1 * np.ones(Movewindow)       # shape   (5,): window class   = +1
y_final = np.r_[y_n0, y_nw]             # shape (105,): combined label vector
```

The label vector `y_final` is fixed throughout all SVM fits: reference observations always carry label −1 and window observations always carry label +1. This two-class formulation turns the monitoring question “is the current window consistent with the reference distribution?” into a classification boundary problem.

5. SVM Model Configuration

A binary `sklearn.svm.SVC` model with RBF kernel is used throughout the pipeline. The same model object is refit at every bootstrap calibration iteration and at every online monitoring step.

```
svm_model = svm.SVC(
    C=0.7, kernel='rbf', gamma=0.4,
    degree=3, coef0=0.0, shrinking=True,
    probability=False, tol=0.001,
    decision_function_shape='ovo',
    random_state=None
)
```

Parameter	Value	Role
C	0.7	Regularisation penalty; controls the trade-off between margin width and misclassification
kernel	'rbf'	Radial basis function kernel; enables nonlinear separation
gamma	0.4	RBF kernel width; larger values give a narrower kernel and more localised boundaries
probability	False	No probability calibration; decision_function values are used directly
decision_function_shape	'ovo'	One-versus-one (has no effect for binary classification; included for explicitness)

Note: The parameters `degree` and `coef0` apply to polynomial kernels and have no effect when `kernel='rbf'`. They are included in the constructor call but are operationally inactive.

6. Monitoring Statistic: D_SVM

The monitoring statistic D_SVM is derived from the SVM decision function via a sigmoid transformation. For a set of window observations, the procedure is:

```
svm_model.fit(y_combine, y_final)              # fit SVM on reference + window
decision_nw = svm_model.decision_function(New_X_w)   # raw decision values
g_nw = 1 / (np.exp(-1 * decision_nw) + 1)     # sigmoid transform
D_SVM_new = np.mean(g_nw)                      # scalar statistic
```

The three computational steps are:

Step 1 — Decision function: The SVM `decision_function` returns the signed distance of each observation from the optimal separating hyperplane. A positive value indicates classification as class +1 (window-like); a negative value indicates class −1 (reference-like).

$$f(x) = w^T\varphi(x) + b \quad \text{(raw decision value for observation x)}$$

Step 2 — Sigmoid transformation: Each decision value is mapped to the interval (0, 1) via the logistic sigmoid:

$$g(x) = 1 / (\exp(-f(x)) + 1)$$

Observations that the SVM classifies firmly as class +1 (window-like, i.e., anomalous) yield $g(x) \to 1$; those classified firmly as class −1 (reference-like, i.e., normal) yield $g(x) \to 0$. Observations near the decision boundary yield $g(x) \approx 0.5$.

Step 3 — Mean aggregation: The scalar statistic D_SVM is the mean of the sigmoid-transformed values over all `Movewindow` observations in the current window:

$$D_SVM = (1/Movewindow) \cdot \Sigma\, g(x_k), \quad k = 1, \ldots, Movewindow$$

D_SVM increases as more window observations are classified as dissimilar from the reference set, indicating process shift. Under in-control conditions, the reference and window observations are drawn from the same distribution, and the SVM decision boundary is uninformative, producing $D_SVM \approx 0.5$.

7. Bootstrap Control Limit Calibration

The control limit `cl_final` is estimated before online monitoring begins, using the bootstrap calibration pool `y_move = y_train_boot[S0:, :]` (900 rows). The calibration procedure applies the same monitoring statistic used online to simulated in-control windows drawn from this pool, and derives the control limit as the mean of 100 upper-quantile estimates.

7.1 Outer Bootstrap Loop (100 replicates)

In each of 100 outer replicates, `y_move` is shuffled in-place by resampling its rows. A sliding window of length `Movewindow` is then passed over the shuffled `y_move`, and D_SVM is computed for each window position:

```
for j in range(100):                                          # outer replicate
    y_move = y_move[np.random.choice(y_move.shape[0],
                                     y_move.shape[0]), :]    # row-shuffle
    for i in range(y_move.shape[0] - Movewindow + 1):        # sliding window
        y_w_move  = y_move[i : Movewindow+i, :]
        y_w_final = np.r_[y_S0, y_w_move]
        svm_model.fit(y_w_final, y_final)
        decision_train = svm_model.decision_function(y_w_move)
        g_nw_train  = 1 / (np.exp(-1 * decision_train) + 1)
        D_SVM_train = np.mean(g_nw_train)
        cl_S0.append(D_SVM_train)                            # accumulate
    cl = np.quantile(cl_S0, 1-a)                             # (1-a) quantile
    cl_1.append(cl)
```

At the end of each outer replicate, the $(1\text{-}\alpha) = 95$th percentile of all accumulated D_SVM values (spanning all windows from all preceding replicates) is appended to `cl_1`. Because `cl_S0` is never reset between outer replicates, the quantile `cl` is computed on the full running accumulation, not just the current replicate's contribution.

7.2 Final Control Limit

```
cl_final = np.mean(cl_1)    # mean of 100 quantile estimates
radius   = cl_final         # operative control limit
```

The operative control limit `radius` is a scalar constant. It does not change during online monitoring.

Symbol	Value	Description
`a`	0.05	Significance level; control limit placed at (1−a) = 95th percentile of bootstrap D_SVM distribution
Outer replicates	100	Number of row-shuffle iterations contributing quantile estimates to cl_1
Bootstrap pool	`y_train_boot[S0:, :]`	900-row calibration pool derived from the 1,000-row bootstrap sample
`cl_final`	Computed	Mean of 100 running-quantile estimates; operative alarm threshold

8. Online Monitoring Loop

8.1 Loop Structure

The monitoring loop iterates over `i = 0, 1, …, X_w.shape[0] - Movewindow`, equal to the number of test observations. At each step, the newest single observation `New_X` and the current window `New_X_w` are extracted, the SVM is refit, and D_SVM is computed:

```
for i in range(X_w.shape[0] - Movewindow + 1):
    New_X   = X_w[Movewindow+i-1 : Movewindow+i, :]   # shape (1, p)
    New_X_w = X_w[i : Movewindow+i, :]                # shape (Movewindow, p)
    y_combine = np.r_[y_S0, New_X_w]                  # shape (105, p)
    svm_model.fit(y_combine, y_final)
    decision_nw = svm_model.decision_function(New_X_w)
    g_nw      = 1 / (np.exp(-1 * decision_nw) + 1)
    D_SVM_new = np.mean(g_nw)
```

8.2 Alarm Decision

The D_SVM statistic is compared to the fixed control limit:

Condition	Action on S0	Action on SVM	Alarm
$\text{D_SVM} > \text{radius}$	Unchanged	Unchanged (will be refit at next step)	Yes; n_alarm incremented
$\text{D_SVM} \leq \text{radius}$	Unchanged	Unchanged	No

Unlike the Time-Adaptive SVDD and RTSVDD pipelines, this algorithm performs no adaptive update of the reference set `y_S0` under any condition. The reference set remains fixed at its initial 100 rows throughout online monitoring. The SVM is refit at every step on the same `y_S0` combined with the current window, so the decision boundary adapts to each new window while the reference distribution remains anchored.

9. Output Collection

```
D_SVM_f = np.array([D_SVM_f]).T              # shape: (n_test, 1)
n_alarm_indicate = np.where(D_SVM_f > radius)[0]  # alarm step indices
radius = np.ones((n_test, 1)) * radius        # broadcast scalar to column
```

Output Variable	Shape	Content
`D_SVM_f`	(n_test, 1)	D_SVM statistic at each test step
`n_alarm`	scalar	Total count of alarm events
`n_alarm_indicate`	(n_alarm,)	Step indices where D_SVM_f > radius

Output Variable	Shape	Content
`radius`	(n_test, 1)	Fixed control limit cl_final broadcast to full test length

10. Visualisation

The function `plot_distance(radius, distance)` renders the D_SVM trajectory against the fixed control limit as a time-series chart.

Element	Style	Represents
Control limit	Red solid line, linewidth 3	`radius`
D_SVM statistic	Black dotted line with circular markers	`D_SVM_f`

The x-axis tick labels map integer indices `[0, 49, 99, 149, 199, 249, 299]` to the date strings `['1-Mar', '20-Apr', '9-Jun', '29-Jul', '17-Sept', '6-Nov', '26-Dec']`. These labels are hardcoded to the reference dataset and should be adjusted for other time series.

11. Hyperparameter Summary

Parameter	Value	Where Used	Role
S0	100	Reference set size	Number of rows in y_S0; fixed throughout pipeline
q1 percentile	0.35	IQR banding on R	Lower quantile bound for selecting reference candidates
q3 percentile	0.50	IQR banding on R	Upper quantile bound for selecting reference candidates
Bootstrap draw size	1,000	y_train_boot construction	Total rows drawn with replacement from the 100-row reference set
Movewindow	5	Window width	Number of observations in each SVM training window and in D_SVM computation
C (SVM)	0.7	All SVM fits	Regularisation penalty for the SVC model
gamma (SVM)	0.4	All SVM fits	RBF kernel width parameter
a (significance)	0.05	Control limit calibration	Control limit placed at (1−a) = 95th percentile of bootstrap

Parameter	Value	Where Used	Role
			D_SVM values
Outer replicates	100	Bootstrap calibration loop	Number of row-shuffle outer iterations

12. Dependencies and Execution

12.1 Required Packages

Package	Usage
numpy	Array operations, random sampling, quantile computation
pandas	CSV ingestion
sklearn.svm.SVC	Binary SVM fitting and decision function computation
sklearn.preprocessing.RobustScaler	Joint normalisation of train and test data
matplotlib / pylab	Control chart visualisation; custom x-axis tick labels

12.2 Execution

Place `real example-traindata.csv` and `real example-testdata.csv` in the working directory and execute:

```
python DSVM.py
```

References

[1] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

[2] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.